PRD: Autonomous Open Source Contribution Finder & Matcher

Version 1.0 (MVP) · Architecture: Next.js full-stack monolith · Status: Ready for build

0. Read First: Rules for the AI Coding Assistant

This document is the single source of truth for the build. Follow these rules in every step:

- Build only what this doc specifies. Do not add authentication, extra pages, an ORM, a state-management library or a UI component library.
- Work on one step of Section 11 at a time. After each step run `npm run build` and `npm run lint`, fix all errors, and stop for review before the next step.
- Use the exact table, column, function, route and environment variable names written here. Never invent new ones.
- Secrets are server-only. Only `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` may reach the browser.
- TypeScript strict mode, no `any`. Keep files small and use the structure in Section 10.
- When something is ambiguous, use the choice in the Locked Decisions table below. If it is not covered, ask before guessing.
- Verify Gemini SDK and Supabase client method names against their current docs before coding; do not rely on memory.

Locked Decisions

These settle the earlier open questions and override anything looser elsewhere in this doc.

Topic	Decision
Framework	Next.js App Router, TypeScript strict, Tailwind, npm, no `src/` folder
Embeddings	`gemini-embedding-001` via the `@google/genai` SDK, 768 dimensions, L2-normalized
Task types	`RETRIEVAL_DOCUMENT` when ingesting, `RETRIEVAL_QUERY` when searching

Topic	Decision
Seed repos	Start with about 10 to 20 active repos listed in `config/repos.ts` (for example `vercel/next.js`, `supabase/supabase`, `fastapi/fastapi`); easy to edit
Issues per repo	Newest 500 open issues (5 pages of 100) per repo, pull requests excluded
Repo language	Fetched once per repo from `GET /repos/{owner}/{repo}` and stored on each issue (the issues endpoint does not return it)
Assigned issues	Skipped at ingestion. There is no `is_claimed` column; ignore that note in Section 5.1
Ingest auth	Accept `Authorization: Bearer <CRON_SECRET>` (Vercel Cron) or header `x-ingest-secret: <INGEST_SECRET>` (manual runs)
Ingest batching	Process 3 repos per request; accept optional `?repo=owner/name`; embed in batches of 50 texts
Search results	Top 20, no pagination in MVP; start `min_similarity` at 0.3 and tune after testing
Caching, analytics, accounts	None in MVP
Rate limiting	Simple in-memory per-IP limit (best effort on serverless), 20 searches per minute
Styling	Tailwind only, system fonts, `lucide-react` for icons is allowed

1. Overview

1.1 Problem

Developers who want to contribute to open source struggle to find issues that fit their skills. GitHub's "good first issue" label is inconsistently applied, goes stale, and says nothing about a person's actual tech stack. Keyword search misses relevant issues that

use different vocabulary (for example, searching "state management" will not surface an issue titled "Redux store not updating on hydration").

1.2 Solution

A web app that embeds open GitHub issues into vectors with Google Gemini and stores them in Supabase (Postgres + pgvector). Users describe their stack or interests in plain language, the app embeds that text, and cosine-similarity search returns the closest open issues instantly, shown as cards with language tags, a difficulty estimate and a direct GitHub link.

1.3 Objectives

- Return relevant issues for a free-text query in under 1.5 seconds (p95).
- Keep a fresh index: issues closed on GitHub disappear from results within 24 hours.
- Ship a usable MVP with zero user accounts, using a single deployable Next.js project.
- Stay within free tiers (Vercel Hobby, Supabase Free, Gemini free tier) for MVP-scale usage.

1.4 Non-goals (MVP)

- User accounts, saved profiles or authentication.
- Tracking PRs, assignments or contribution history.
- Automatically commenting on or claiming issues on GitHub.
- Non-GitHub sources (GitLab, Codeberg).
- Fine-tuned ranking models.

2. Target Users and Use Cases

Persona	Need
Beginner developer	Find approachable issues in tools they already use
Mid-level developer building a portfolio	Find meaningful issues in a specific stack (e.g. "TypeScript, tRPC, Postgres")
Student / hackathon / Hacktoberfest participant	Find many small, well-scoped issues quickly

Persona	Need
Maintainer (secondary)	Gain visibility for under-served issues

Primary user stories:

1. As a developer, I type "I know React and TypeScript, want to fix accessibility bugs" and see matching open issues ranked by relevance.
2. As a developer, I filter results by programming language and difficulty.
3. As a developer, I click a card and land on the GitHub issue page.
4. As the site owner, I trigger or schedule ingestion for a list of repositories without manual SQL work.

3. Technology Stack

Layer	Choice	Notes
Framework	Next.js (App Router), TypeScript, Tailwind CSS	Server Actions and Route Handlers for backend logic
Database	Supabase Postgres with `pgvector`	HNSW index, cosine distance
Embeddings	Google Gemini `gemini-embedding-001` (or `gemini-embedding-2`)	Output dimensionality set to 768
Data source	GitHub REST API	Authenticated with a personal access token
Hosting	Vercel + Supabase	Vercel Cron for scheduled ingestion

4. System Architecture and Data Flow

Ingestion path (write): Cron or admin trigger → `/api/ingest` route → GitHub REST API (list open issues per repo) → clean and build embedding text → Gemini embeddings (document mode, batched) → upsert into Supabase `issues` using the service-role key.

Search path (read): User types query → Server Action `searchIssues` → Gemini embedding (query mode) → Supabase RPC `match_issues` (cosine similarity, filters) → ranked results returned to the UI.

Design principles:

- All secrets and all writes live on the server. The browser never sees the service-role key, the Gemini key or the GitHub token.
- The browser only talks to the app's own Server Actions or Route Handlers, which in turn call Supabase and Gemini.
- Ingestion is idempotent: re-running it never creates duplicates (upsert on `url` or `github_id`).

5. Functional Requirements

5.1 Ingestion Worker

ID	Requirement	Priority
ING-1	Accept a list of target repos (`owner/name`), stored in a config file or a `repos` table	Must
ING-2	Fetch open issues via `GET /repos/{owner}/{repo}/issues?state=open&per_page=100`, following pagination	Must
ING-3	Exclude pull requests (the issues endpoint also returns PRs; they carry a `pull_request` key)	Must
ING-4	Build embedding input as `repo + title + labels + first ~1,500 chars of body` (strip markdown, HTML comments, issue templates, images)	Must
ING-5	Generate embeddings with Gemini using document task type, 768 dimensions, batched requests	Must
ING-6	Upsert into `issues` keyed on `url`; skip re-embedding when a `content_hash` is unchanged	Must
ING-7	Mark issues closed or missing on GitHub as `state = 'closed'` (or delete) during each sync	Must
ING-8	Protect the route with a secret (`CRON_SECRET` / `INGEST_SECRET` header check)	Must

ID	Requirement	Priority
ING-9	Process in chunks that fit Vercel's function time limit; resume via a cursor or by repo	Should
ING-10	Log per-run stats: repos processed, issues added/updated/closed, errors	Should
ING-11	Scheduled run via Vercel Cron (daily)	Should

Ingestion rules:

- Respect GitHub rate limits (5,000 requests/hour authenticated). Read `X-RateLimit-Remaining` and back off or stop gracefully when near zero.
- Use conditional requests (`If-None-Match` / `since` parameter) on later runs to cut API usage.
- Skip issues that are locked, already assigned, or have a linked open PR when that information is available (flag as `is_claimed`).

5.2 Semantic Matcher

ID	Requirement	Priority
SEM-1	Convert the user's query into a 768-dim embedding using Gemini query task type	Must
SEM-2	Call a Postgres function `match_issues` that orders by cosine distance (`<=>`) and returns the top K (default 20)	Must
SEM-3	Return similarity score (`1 - distance`) with each result	Must
SEM-4	Support optional filters: language, difficulty, minimum similarity threshold	Must
SEM-5	Only return issues with `state = 'open'`	Must
SEM-6	Empty query returns the most recently updated issues (no embedding call)	Should
SEM-7	Cache query embeddings for repeated queries (in-memory or a small `query_cache` table)	Could

ID	Requirement	Priority
SEM-8	Rate-limit the search endpoint per IP to protect the Gemini quota	Should

5.3 Difficulty Score

MVP uses a transparent heuristic, computed at ingestion and stored as an integer 1 to 5 plus a label (Beginner, Easy, Medium, Hard):

- Labels such as `good first issue`, `help wanted`, `documentation`, `typo` push the score down.
- Labels such as `bug` with long bodies, `performance`, `architecture`, `security` push it up.
- Body length, number of comments and the age of the issue are minor adjustments.

The difficulty is clearly shown as an estimate. Phase 2 may use a Gemini text model to classify difficulty.

5.4 Frontend Dashboard

ID	Requirement	Priority
UI-1	Homepage with a prominent search bar and example prompts ("Python + FastAPI, API bugs")	Must
UI-2	Language filter chips (JavaScript, TypeScript, Python, Go, Rust, Java, etc.) that combine with the search	Must
UI-3	Responsive grid of issue cards: title, repo name, language tag, difficulty badge, match percentage, short description snippet, "Open on GitHub" link (new tab)	Must
UI-4	Loading skeletons, empty state ("No matches, try broader terms") and error state	Must
UI-5	Debounced or submit-based search so each keystroke does not trigger an embedding call	Must

ID	Requirement	Priority
UI-6	Difficulty filter and a "load more" or pagination control	Should
UI-7	Dark mode, keyboard accessibility, visible focus states, semantic HTML	Should
UI-8	Persist the last query and filters in URL query params so searches are shareable	Should

6. Database Design

The original schema is a good base. Recommended improvements are marked below: a `github_id`, `state`, labels, a content hash for skip-if-unchanged logic, a difficulty score, timezone-aware timestamps, and row-level security.

```
create extension if not exists vector;

create table issues (
  id uuid primary key default gen_random_uuid(),
  github_id bigint unique,
  title text not null,
  description text,
  repo_name text not null,
  url text not null unique,
  language text,
  labels text[] default '{}',
  state text not null default 'open',
  comments_count int default 0,
  difficulty int check (difficulty between 1 and 5),
  content_hash text,
  embedding vector(768),
  github_created_at timestamptz,
  github_updated_at timestamptz,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

create index on issues using hnsw (embedding vector_cosine_ops);
create index on issues (state, language);
create index on issues (repo_name);
```

```
-- Security: public can read open issues, only the service role writes
alter table issues enable row level security;
create policy "public read open issues" on issues
  for select using (state = 'open');

-- Semantic search function
create or replace function match_issues(
  query_embedding vector(768),
  match_count int default 20,
  filter_language text default null,
  max_difficulty int default null,
  min_similarity float default 0.0
)
returns table (
  id uuid, title text, description text, repo_name text, url text,
  language text, labels text[], difficulty int, similarity float
)
language sql stable
as $$
  select i.id, i.title, i.description, i.repo_name, i.url,
         i.language, i.labels, i.difficulty,
         1 - (i.embedding <=> query_embedding) as similarity
  from issues i
  where i.state = 'open'
    and (filter_language is null or i.language = filter_language)
    and (max_difficulty is null or i.difficulty <= max_difficulty)
    and 1 - (i.embedding <=> query_embedding) >= min_similarity
  order by i.embedding <=> query_embedding
  limit match_count;
$$;
```

Important notes on the vector column:

- `gemini-embedding-001` outputs 3072 dimensions by default, but supports truncated output through the `outputDimensionality` parameter. Request 768 explicitly on every call (ingestion and search) so dimensions always match the column.
- Embeddings at dimensions below 3072 should be L2-normalized before storing or comparing. Normalize in code immediately after each API response.
- Use the same embedding model for documents and queries. Changing models requires re-embedding the whole table.

- If `gemini-embedding-2` is chosen instead, check its supported dimensions and task-type options in the current Gemini docs and keep the column size consistent.

7. API and Server Action Contracts

Endpoint / Action	Method	Purpose	Auth
`searchIssues(query, filters)`	Server Action	Embed query, call `match_issues`, return results	Public, rate-limited
`/api/ingest`	POST/GET	Run ingestion for configured repos	Secret header
`/api/health`	GET	Check DB and external API connectivity	Public (no secrets)

Search request: `{ query: string (max 500 chars), language?: string, maxDifficulty?: 1-5, limit?: number (max 50) }`

Search response: `{ results: [{ id, title, repoName, url, language, labels, difficulty, similarity, snippet }], tookMs }`

Error behavior: validation errors return a friendly message; Gemini or Supabase failures return a generic error and are logged server-side with no secret leakage.

8. Non-Functional Requirements

Area	Requirement
Performance	Search p95 under 1.5 s (Gemini embed ≈ 200 to 500 ms plus indexed query); page loads fast with server-rendered shell
Scalability	HNSW index comfortably serves hundreds of thousands of issues; MVP target 10k to 50k issues
Reliability	Ingestion is idempotent and resumable; one failing repo does not abort the run

Area	Requirement
Security	Secrets only in server environment; RLS on; service-role key never exposed; ingest route protected; input length limits; no `NEXT_PUBLIC_` prefix on secrets
Privacy	No user accounts or personal data stored; optionally log anonymous queries for analytics
Accessibility	WCAG 2.1 AA targets: contrast, keyboard navigation, ARIA labels on chips
Observability	Structured logs for ingestion and search; Vercel analytics; error tracking (e.g. Sentry) is optional
Cost	Free-tier friendly; embed only new or changed issues

9. Environment Variables

```
NEXT_PUBLIC_SUPABASE_URL=your_supabase_project_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key   # safe for browser,
RLS-protected
SUPABASE_SERVICE_ROLE_KEY=your_supabase_service_role_key  # server only
GEMINI_API_KEY=your_google_gemini_api_key                 # server only
GITHUB_TOKEN=your_github_personal_access_token            # server only,
public_repo read scope
INGEST_SECRET=long_random_string                          # protects
/api/ingest
CRON_SECRET=long_random_string                            # Vercel Cron
authorization
```

Rules: never commit `.env.local`; add the same variables in the Vercel project settings; rotate keys if exposed.

10. Suggested Project Structure

```
/app
  page.tsx                  # dashboard (search, chips, grid)
  actions.ts                # searchIssues server action
  /api/ingest/route.ts      # ingestion worker
  /api/health/route.ts
/components
  SearchBar.tsx  LanguageChips.tsx  IssueCard.tsx  IssueGrid.tsx
Skeletons.tsx
/lib
  supabase/server.ts        # service-role client (server only)
  supabase/client.ts        # anon client
  gemini.ts                 # embed(), normalize(), batching
  github.ts                 # fetchOpenIssues(), pagination, rate-limit
handling
  difficulty.ts             # heuristic scoring
  cleanText.ts              # markdown/HTML stripping
/config/repos.ts            # target repositories
/supabase/migrations/       # SQL schema and match_issues function
```

11. Build Plan (Vibe-Coding Backlog)

Run these prompts in order in Cursor or Windsurf, and verify each step before moving on.

Step 1: Project setup and layout. "Initialize a Next.js App Router TypeScript project with Tailwind CSS. Build the homepage: a centered search input, language filter chips, example query suggestions and a responsive grid of IssueCard components using mock data. Include skeleton loaders and an empty state." *Done when:* the page renders on mobile and desktop with mock data.

Step 2: Database connection. "Install `@supabase/supabase-js`. Create a server-only client using the service-role key and a browser client using the anon key. Run the SQL migration, then write `getRecentIssues()` and a typed `Issue` interface." *Done when:* the homepage lists real rows inserted manually.

Step 3: Ingestion and embedding pipeline. "Create `/api/ingest` that is protected by an `INGEST_SECRET` header. For each repo in `config/repos.ts`, fetch open issues from the GitHub REST API with pagination, drop pull requests, clean the text, compute a content hash and difficulty score, embed new or changed issues with Gemini (768 dims, document

task type, normalized, batched) and upsert into Supabase. Mark issues no longer open as closed. Handle rate limits and per-repo errors." *Done when:* running the route populates 500+ issues without duplicates, and a second run embeds nothing new.

Step 4: Semantic search and frontend integration. "Add the `match_issues` SQL function. Write a `searchIssues` Server Action that validates input, embeds the query with Gemini (query task type, 768 dims, normalized) and calls the RPC with language and difficulty filters. Connect it to the search bar and chips with a loading state and URL params." *Done when:* the query "accessibility bugs in React components" returns relevant React issues with match percentages.

Step 5: Hardening and launch. "Add per-IP rate limiting to search, input validation, error boundaries, Vercel Cron for daily ingestion, a health endpoint and basic SEO metadata." Then deploy to Vercel and set all environment variables.

12. Testing and QA

- **Unit tests:** text cleaning, difficulty heuristic, normalization, GitHub pagination parsing.
- **Integration tests:** ingestion against a small known repo, idempotency check (run twice, expect zero duplicates), `match_issues` with filters.
- **Relevance evaluation:** curate about 30 queries with expected repos or issues and track top-5 hit rate whenever the embedding text format changes.
- **Manual QA:** mobile layout, keyboard-only navigation, empty and error states.

13. Risks and Mitigations

Risk	Impact	Mitigation
Vercel function timeout during large ingestion	Incomplete sync	Process a few repos per invocation; cursor-based resume; chunked cron calls
Gemini rate limits or free-tier quota	Search or ingestion failures	Batch requests, retry with backoff, cache query embeddings, rate-limit users
GitHub API rate limits	Missed updates	Authenticated token, conditional requests, `since` parameter

Risk	Impact	Mitigation
Stale or already-claimed issues	Poor user experience	Daily sync, close detection, skip assigned issues
Noisy issue text lowers match quality	Weak relevance	Strip templates and markdown, include labels and repo name, evaluate with a query set
Dimension or model mismatch	Query errors	Single `EMBEDDING_DIM` constant, same model for documents and queries
Key leakage	Security incident	Server-only keys, no `NEXT_PUBLIC_` on secrets, RLS enabled
Spam or abuse of the search endpoint	Cost and quota burn	Per-IP rate limit, input length cap

14. Code Contracts (Reference Skeletons)

Use these as the shape of the code, not as finished files. Adapt, but keep the names and signatures.

14.1 Shared types and constants (`lib/types.ts`)

```
export const EMBEDDING_MODEL = "gemini-embedding-001";
export const EMBEDDING_DIM = 768;

export type Difficulty = 1 | 2 | 3 | 4 | 5;

export interface IssueRow {
  github_id: number;
  title: string;
  description: string | null;
  repo_name: string;
  url: string;
  language: string | null;
  labels: string[];
  state: "open" | "closed";
  comments_count: number;
  difficulty: Difficulty;
```

```
  content_hash: string;
  embedding: number[];
  github_created_at: string;
  github_updated_at: string;
}

export interface SearchFilters {
  language?: string;
  maxDifficulty?: Difficulty;
  limit?: number; // default 20, max 50
}

export interface SearchResult {
  id: string;
  title: string;
  repoName: string;
  url: string;
  language: string | null;
  labels: string[];
  difficulty: Difficulty;
  similarity: number; // 0 to 1
  snippet: string;    // first ~200 chars of description
}
```

14.2 Embeddings (`lib/gemini.ts`)

```
import { GoogleGenAI } from "@google/genai";
import { EMBEDDING_MODEL, EMBEDDING_DIM } from "./types";

const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY! });

export function normalize(v: number[]): number[] {
  const norm = Math.sqrt(v.reduce((s, x) => s + x * x, 0)) || 1;
  return v.map((x) => x / norm);
}

export async function embedTexts(
  texts: string[],
  taskType: "RETRIEVAL_DOCUMENT" | "RETRIEVAL_QUERY"
): Promise<number[][]> {
  const out: number[][] = [];
  for (let i = 0; i < texts.length; i += 50) {
```

```
    const batch = texts.slice(i, i + 50);
    const res = await ai.models.embedContent({
      model: EMBEDDING_MODEL,
      contents: batch,
      config: { taskType, outputDimensionality: EMBEDDING_DIM },
    });
    for (const e of res.embeddings ?? []) out.push(normalize(e.values ??
[]));
  }
  if (out.length !== texts.length) throw new Error("Embedding count
mismatch");
  return out;
}
```

14.3 GitHub fetching (`lib/github.ts`)

```
const headers = {
  Authorization: `Bearer ${process.env.GITHUB_TOKEN}`,
  Accept: "application/vnd.github+json",
  "X-GitHub-Api-Version": "2022-11-28",
};

// 1. GET /repos/{repo} -> language
// 2. GET /repos/{repo}/issues?state=open&per_page=100&page=N (max 5 pages)
// 3. Drop items with a `pull_request` key or a non-empty `assignee`
// 4. Stop early if the X-RateLimit-Remaining header is below 50
export async function fetchOpenIssues(repo: string): Promise<GithubIssue[]> {
/* ... */ }
```

Embedding text format (keep stable; changing it means re-embedding everything):

```
Repo: {repo_name}
Language: {language}
Labels: {labels joined by comma}
Title: {title}
Body: {cleaned body, first 1500 chars}
```

14.4 Search action (`app/actions.ts`)

```
"use server";

export async function searchIssues(
  query: string,
  filters: SearchFilters = {}
): Promise<{ results: SearchResult[]; tookMs: number }> {
  // 1. Trim query, reject if longer than 500 characters
  // 2. Apply per-IP rate limit (headers().get("x-forwarded-for"))
  // 3. If query is empty: return the 20 most recently updated open issues,
no embedding call
  // 4. const [vec] = await embedTexts([query], "RETRIEVAL_QUERY");
  // 5. supabase.rpc("match_issues", {
  //      query_embedding: vec,
  //      match_count: filters.limit ?? 20,
  //      filter_language: filters.language ?? null,
  //      max_difficulty: filters.maxDifficulty ?? null,
  //      min_similarity: 0.3,
  //    })
  // 6. Map rows to SearchResult, return with elapsed milliseconds
}
```

14.5 Ingestion route (`app/api/ingest/route.ts`)

Flow per request: verify auth, pick up to 3 repos (or the one in `?repo=`), and for each repo fetch issues, compute `content_hash` (SHA-256 of the embedding text), load existing hashes for those URLs from Supabase, embed only new or changed issues, upsert on `url`, then set `state = 'closed'` for that repo's stored open issues that no longer appear in GitHub's open list. Return JSON `{ repos, added, updated, closed, errors }`. One failing repo must not stop the others.

15. Definition of Done (MVP)

- [ ] `npm run build` and `npm run lint` pass with no errors or warnings
- [ ] SQL migration runs cleanly on a fresh Supabase project, including RLS and `match_issues`
- [ ] Ingesting the seed repos stores 500+ issues with 768-dimension embeddings and no duplicate URLs
- [ ] Running ingestion a second time embeds zero unchanged issues

- [ ] Closed GitHub issues are marked `closed` and no longer appear in search
- [ ] Query "accessibility bugs in React components" returns relevant React or frontend issues at the top
- [ ] Language chips and the search box combine correctly
- [ ] Empty, loading and error states all render
- [ ] Layout works at 375 px and 1280 px widths
- [ ] No secret appears in client bundles, logs or git history; `.env.local` is gitignored
- [ ] `/api/ingest` returns 401 without the secret
- [ ] Deployed to Vercel with all environment variables set and a daily cron configured

16. Common Pitfalls to Avoid

- Using the service-role key in any file that is imported by a client component.
- Mixing embedding dimensions or models between ingestion and search (queries then fail or return nonsense).
- Forgetting to normalize vectors after requesting 768 dimensions.
- Embedding on every keystroke; search only on submit or after a debounce of 500 ms or more.
- Treating GitHub pull requests as issues.
- Running a long ingestion in one request and hitting Vercel's function time limit; stick to 3 repos per call.
- Passing a query string to the `vector` column instead of an embedding array.
- Putting the `NEXT_PUBLIC_` prefix on any secret.